📚 Postman Testing - Complete Documentation Summary

📖 Overview

I've created **4 comprehensive testing documents** for your Trading Platform API. Each serves a different purpose:

File	Purpose	Best For
`POSTMAN_API_TESTS.md`	Complete API documentation with examples	Learning & Understanding
`POSTMAN_QUICK_REFERENCE.md`	Quick lookup table & tips	Quick Reference
`CURL_COMMANDS.md`	Terminal/CLI testing commands	Automation & Scripting
`Trading_Platform_API.postman_collection.json`	Importable Postman collection	Postman GUI Testing

📁 Files Created

```
c:\Project\TradingSys\Ts-backend\
├── POSTMAN_API_TESTS.md                    (Comprehensive documentation)
├── POSTMAN_QUICK_REFERENCE.md              (Quick lookup guide)
├── CURL_COMMANDS.md                         (Terminal commands)
└── Trading_Platform_API.postman_collection.json (Postman import)
```

🚀 Quick Start Guide

Option 1: Using Postman GUI (Recommended for Most Users)

Step 1: Import Collection

```
1. Open Postman
2. Click "Import" button (top-left)
3. Select file: Trading_Platform_API.postman_collection.json
4. Collection will appear in left sidebar
```

Step 2: Set Environment Variables

```
{
  "base_url": "http://localhost:8000",
  "user_id": "1",
  "symbol": "SBIN"
}
```

Step 3: Run Tests

```
1. Expand "Trading Platform API" collection
2. Click "Users API" folder
3. Click "1. Create User" request
4. Click "Send" button
5. View response in lower panel
```

Option 2: Using CURL Commands (For Terminal/Automation)

Step 1: Open Terminal

```
# Windows PowerShell, Mac Terminal, or Linux bash
cd c:\Project\TradingSys\Ts-backend
```

Step 2: Run CURL Command

```
# Example: Create user
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Test User",
    "email": "test@example.com"
  }'
```

Step 3: View Response

```
{
  "id": 1,
  "name": "Test User",
  "email": "test@example.com",
  "created_at": "2026-04-19T14:15:00.000Z",
  "updated_at": "2026-04-19T14:15:00.000Z"
}
```

Option 3: Using API Documentation (For Learning)

Step 1: Read POSTMAN_API_TESTS.md

- Shows all 13 API endpoints
- Includes request/response examples
- Explains parameters and error cases

Step 2: Check POSTMAN_QUICK_REFERENCE.md

- Quick summary table
- Status codes and meanings
- Common errors and solutions

📋 Complete API Checklist

☑ All 13 APIs Documented

Users API (4 endpoints)

- ☑ 1. Create User - POST /users
- ☑ 2. Get User - GET /users/{user_id}
- ☑ 3. List Users - GET /users?skip=0&limit=10
- ☑ 4. Delete User - DELETE /users/{user_id}

Orders API (4 endpoints)

- ☑ 5. Create Order (BUY/SELL) - POST /orders
- ☑ 6. Get Order History - GET /orders/{user_id}
- ☑ 7. Get Order Count - GET /orders/{user_id}/count

Portfolio API (3 endpoints)

- ☑ 8. Get Complete Portfolio - GET /portfolio/{user_id}
- ☑ 9. Get User Positions - GET /portfolio/{user_id}/positions
- ☑ 10. Get Wallet Balance - GET /portfolio/{user_id}/balance

Market API (3 endpoints)

- ☑ 11. Get All Prices - GET /market/prices
- ☑ 12. Get Symbol Price - GET /market/price/{symbol}
- ☑ 13. Market Health - GET /market/health

🎯 Test Scenarios Covered

☑ Success Cases

- ✓ Create user with valid data
- ✓ Get existing user/orders/portfolio
- ✓ List users with pagination
- ✓ Place BUY order with sufficient balance
- ✓ Place SELL order with sufficient quantity
- ✓ Get market prices
- ✓ WebSocket notifications

☑ Error Cases

- ✓ Duplicate email (400)
- ✓ Non-existent user (404)
- ✓ Insufficient balance for BUY (400)
- ✓ Insufficient quantity for SELL (400)
- ✓ Invalid order side (400)
- ✓ Invalid symbol (404)
- ✓ Redis not available (503)

📊 Testing Workflow

```
START
  ↓
[1] Create User
  ├─ POST /users
  └─ Save user_id
  ↓
[2] Check Balance
  ├─ GET /portfolio/{user_id}/balance
  └─ Verify: ₹10,00,000
  ↓
[3] Get Prices
  ├─ GET /market/prices
  └─ Save symbol prices
  ↓
[4] Place BUY Order
  ├─ POST /orders (BUY)
  └─ Verify: balance decreased, position increased
  ↓
[5] Check Portfolio
  ├─ GET /portfolio/{user_id}
  └─ Verify: holdings updated, P&L calculated
  ↓
[6] View Order History
  ├─ GET /orders/{user_id}
  └─ Verify: order appears in history
  ↓
[7] Place SELL Order
  ├─ POST /orders (SELL)
```

```
    └─ Verify: position reduced, balance increased
    ↓
  [8] Final Checks
    ├─ GET /portfolio/{user_id}
    └─ GET /orders/{user_id}/count
    ↓
  END ✓
```

🔑 Key Information

Base URL

```
http://localhost:8000
```

Available Symbols

```
SBIN, RELIANCE, TCS, INFY, HDFC
```

Initial Wallet Balance

```
₹10,00,000 (10 Lakh Rupees)
```

Default Status Codes

```
200: Success (GET)
201: Created (POST)
204: Deleted (DELETE)
400: Bad Request (Invalid data)
404: Not Found (Resource doesn't exist)
500: Server Error
503: Service Unavailable (Redis down)
```

💻 System Requirements

For Postman Testing

- Postman Desktop App (free)
- Running Trading Platform API on port 8000

For CURL Testing

- Terminal/PowerShell
- `curl` command available (pre-installed on Mac/Linux, needs download on Windows)
- `jq` (optional, for JSON formatting)

Running the API

```
cd c:\Project\TradingSys\Ts-backend
uvicorn app.main:app --reload
```

Check if API is Running

```
curl http://localhost:8000/market/health
```

📝 Postman Collection Import

Method 1: Direct Import

```
1. Open Postman
2. File → Import
3. Select: Trading_Platform_API.postman_collection.json
4. Click "Import"
```

Method 2: Drag & Drop

```
1. Open Postman
2. Drag Trading_Platform_API.postman_collection.json into Postman
3. Select "Import as a collection"
```

Method 3: Import from URL

```
Not applicable - use local file
```

🧪 Test Execution

In Postman:

Single Request:

```
1. Select API endpoint
2. Click "Send"
3. View response
```

Multiple Requests in Sequence:

```
1. Click "Runner" button
2. Select Collection
3. Click "Run"
4. View test results
```

With Tests/Assertions:

```
Each endpoint has pre-configured tests that validate:
- Status code
- Response schema
- Required fields
- Data types
```

In Terminal:

Single Command:

```
curl -X GET http://localhost:8000/market/prices
```

Multiple Commands:

```
bash CURL_COMMANDS.md

# Or create a script file and run:
./test_api.sh
```

📊 Response Examples

User Created (201)

```
```

```json
{
  "id": 1,
  "name": "Aaryan Kumar",
  "email": "aaryan@example.com",
  "created_at": "2026-04-19T14:15:00.000Z",
  "updated_at": "2026-04-19T14:15:00.000Z"
}
```

Order Executed (201)

```json
{
  "id": 1,
  "user_id": 1,
  "symbol": "SBIN",
  "quantity": 10,
  "price": 450.75,
  "total_amount": 4507.50,
  "side": "BUY",
  "status": "COMPLETED",
  "created_at": "2026-04-19T14:15:30.000Z",
  "updated_at": "2026-04-19T14:15:30.000Z"
}
```

Portfolio (200)

```json
{
  "user_id": 1,
  "wallet_balance": 995492.50,
  "holdings": [
    {
      "symbol": "SBIN",
      "quantity": 10,
      "average_price": 450.75,
      "current_price": 455.20,
      "total_invested": 4507.50,
      "current_value": 4552.00,
      "unrealized_pnl": 44.50,
      "pnl_percentage": 0.99
    }
  ],
  "total_portfolio_value": 1000044.50,
  "total_invested": 4507.50,
  "total_unrealized_pnl": 44.50,
  "total_pnl_percentage": 0.99
}
```

🔍 Validation Points

Data Validation

- ☑ Email format validated
- ☑ Quantity must be > 0
- ☑ Side must be "BUY" or "SELL"
- ☑ Symbol must be valid
- ☑ User must exist
- ☑ Balance validation for BUY
- ☑ Quantity validation for SELL

Business Logic

- ☑ Wallet balance deducted on BUY
- ☑ Wallet balance increased on SELL
- ☑ Position created on first BUY
- ☑ Position quantity updated on subsequent trades
- ☑ Weighted average price calculated correctly
- ☑ P&L calculated correctly
- ☑ Order history maintained

Database Constraints

- ☑ Duplicate email prevention
- ☑ User/position unique constraint (user_id + symbol)
- ☑ Foreign key relationships
- ☑ Cascade delete for related records

🚨 Troubleshooting

Issue	Solution
"Port 8000 already in use"	Kill process on port 8000 or use different port
"Database connection failed"	Verify MySQL is running and credentials are correct
"No prices available"	Start Redis: `docker run -d -p 6379:6379 redis`
"Postman can't connect"	Verify API is running: `curl http://localhost:8000/market/health`
"Invalid email format"	Use valid email like `user@example.com`
"Insufficient balance"	Initial balance is ₹10,00,000, adjust quantity accordingly

---

📚 Documentation Files

POSTMAN_API_TESTS.md

Contains:

- All 13 API endpoints
- Request/response examples
- Error scenarios
- Test cases for each API
- Complete test sequence
- Postman setup instructions
- Validation checklist

Use When:

- Learning the API
- Implementing tests
- Understanding response formats
- Checking error handling

POSTMAN_QUICK_REFERENCE.md

Contains:

- API summary table
- Status codes
- Key parameters
- Default values
- Postman setup steps
- Test flow diagram
- Common errors & solutions

Use When:

- Need quick lookup
- Forgotten exact endpoint
- Need error explanation
- Quick testing reference

CURL_COMMANDS.md

Contains:

- All CURL command examples
- Error scenario tests
- Batch test script
- Advanced curl features
- Performance testing
- Debugging tips
- Stress test examples

Use When:

- Testing via terminal
- Automating tests
- CI/CD integration
- Performance testing
- Scripting/batch operations

Trading_Platform_API.postman_collection.json

Contains:

- Importable Postman collection
- 13 API endpoints
- Pre-configured tests
- Environment variables
- Error test cases

Use When:

- Want GUI testing
- Need organized collection
- Want automated test validation
- Sharing tests with team

✨ Key Features Documented

For Each API Endpoint:

1. **Exact URL** - Copy-paste ready
2. **HTTP Method** - GET, POST, DELETE
3. **Request Body** - JSON with all required fields
4. **Response Body** - Success response example
5. **Error Responses** - Common error cases
6. **Status Code** - Expected HTTP status
7. **Test Cases** - What to test
8. **Parameters** - All required/optional params
9. **Validation Rules** - Constraints & rules
10. **Business Logic** - How it works

🎓 Learning Path

For Beginners:

1. Read: POSTMAN_API_TESTS.md (Overview section)
2. Import: Trading_Platform_API.postman_collection.json
3. Test: Follow "Complete Test Sequence" in POSTMAN_API_TESTS.md
4. Check: POSTMAN_QUICK_REFERENCE.md for quick lookup

For Advanced Users:

1. Read: CURL_COMMANDS.md
2. Write: Custom test scripts
3. Automate: Performance & load testing
4. Integrate: CI/CD pipeline

📞 Support & Help

If API Not Responding:

1. Check if server is running: `http://localhost:8000/market/health`
2. Check MySQL: Verify database connection
3. Check Redis: Verify Redis is running for prices
4. Check logs: View server terminal for errors

If Test Fails:

1. Check user_id exists: `GET /users/{user_id}`
2. Check balance: `GET /portfolio/{user_id}/balance`
3. Check position: `GET /portfolio/{user_id}/positions`
4. View error message carefully

🎉 Summary

You now have **complete testing documentation** including:

- ☑ **13 API endpoints** fully documented
- ☑ **Request/Response examples** for each
- ☑ **Error scenarios** with solutions
- ☑ **Postman collection** ready to import
- ☑ **CURL commands** for terminal testing
- ☑ **Quick reference guide** for fast lookup
- ☑ **Complete test workflow** to follow

Ready to test! 🚀

Last Updated: April 19, 2026
API Version: 1.0.0
Documentation Version: 1.0.0